SOFTWARE ENGINEERING · CHAPTER 11

Lab-11: Component Diagrams

You take the ITC Campus Library System from Labs 07-10 (use cases, class diagram, activity and sequence diagrams) and structure it as a modular monolith: identify components and their interfaces, draw the UML 2.5 component diagram with ports, lollipops and sockets, redesign the Loans component in hexagonal style, map the components to a Maven multi-module build with `module-info.java` and an ArchUnit guard, then validate dependency direction and document the decisions with ADRs and a C4 container diagram. The challenge extracts Notifications into its own service behind a message broker without breaking the monolith.

UML 2.5.1 · PlantUML · C4 JDK 25 · Maven 3.9.16 · Spring Boot 4.1.1 ArchUnit 1.4.1 · Spring Modulith ≈ 5 hours

5 tasks + 1 challenge

How to work through this lab. Do the tasks in order; each builds on the previous. Read the *Goal*, follow the *Steps*, compare with the *Expected output*, and tick the *Acceptance checklist* (saved in your browser). Open a hint only when stuck for more than 10 minutes. Diagrams may be drawn in PlantUML, draw.io or Mermaid-compatible tools, but you must submit the **source text** and a **PNG or SVG export** of every diagram. All Java in Tasks 4-5 and the challenge must compile with `mvn -q verify`.

0 Setup — 20 min

1. Verify the tool chain (PlantUML needs Java and, for `package`/C4 layouts, Graphviz):

```
java -version        # openjdk 25
mvn -version         # Apache Maven 3.9.16
dot -V               # graphviz version 12.x  (winget install graphviz / apt install graphviz)
curl -L -o plantuml.jar https://github.com/plantuml/plantuml/releases/latest/download/plantuml.jar
java -jar plantuml.jar -version
# render every diagram in a folder to PNG:
java -jar plantuml.jar -tpng diagrams/*.puml
```

2. Create the lab folder next to your Lab-10 work and bring over the artefacts you will read from: the Lab-08 class diagram (`library-classes.puml`) and the Lab-10 sequence diagrams (`borrow-book.puml`, `return-book.puml`, `reserve-book.puml`).

```
lab-11/
├── diagrams/            .puml sources + exported .png  (Tasks 1, 2, 3, 5,
challenge)
├── docs/
│   ├── adr/             ADR-001.md, ADR-002.md  (Task 5)
│   └── review/          dependency-review.md    (Task 5)
├── inputs/              library-classes.puml, borrow-book.puml, ... (copied
from Labs 08-10)
└── library/             Maven reactor (Task 4 onwards)
    ├── pom.xml
    ├── library-catalog/
    ├── library-loans/
    ├── library-notifications/
    └── library-app/
```

3. The Maven modules are generated in Task 4. Dependencies you will need there (versions from the course baseline; Spring Boot's parent manages the rest):

```
<!-- library/pom.xml, <dependencyManagement> extras -->
<dependency>
  <groupId>com.tngtech.archunit</groupId>
  <artifactId>archunit-junit5</artifactId>
  <version>1.4.1</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.modulith</groupId>
  <artifactId>spring-modulith-bom</artifactId>
  <version>2.0.1</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>
```

4. If you did not keep your Lab-08/10 sources, use the reference inputs given in the starter of Task 1: the class list and the "Borrow a book" sequence are enough to complete every task.

1 Identify components and interfaces Easy 40 min

GOAL

Zoom out from the class diagram: group the classes of the library system into 6-8 components, and for each component list what it **provides** and what it **requires**, derived from the messages in the sequence diagrams.

STEPS

1. Open `inputs/library-classes.puml` (or the reference class list in the starter). Group the classes into components by the rule *"changes for the same reason, owns the same data"*. You must end with 6-8 components; the expected set is **Web UI, Catalogue, Loans, Members, Reservations, Notifications, Identity** (Reporting optional). Write the grouping into `docs/components.md` as a table *Component · Classes · Owned tables*.
2. Walk the "Borrow a book" and "Return a book" sequence diagrams from Lab-10. Every message that crosses from one component to another becomes an **interface operation**. Record them in a second table *From component · To component · Operation · Interface name*.
3. Name the interfaces after the *role*, not the class: `LoanApi` (place/return a loan), `LoanQuery`, `CatalogLookup`, `CopyAvailability`, `MemberLookup`, `ReservationApi`, `Notifier`, `AuthApi`. One interface may have 2-6 operations; if you have more than 6, split it.
4. For every component write two lists in `components.md`: **Provided** (interfaces it implements) and **Required** (interfaces it calls). Every required interface must appear as provided on exactly one other component, or be marked *external*.
5. Write `diagrams/task1-components.puml`: each component with its provided interfaces as `interface` + `I - C` lines and its required interfaces as `C ..> I : uses`. No layout tuning yet, no databases.
6. Render with `java -jar plantuml.jar -tpng diagrams/task1-components.puml` and check that the PNG shows every interface exactly once as a lollipop.

STARTER: REFERENCE INPUTS FROM LABS 08 AND 10

```
Classes (Lab-08 class diagram, package edu.itc.library):
  Book, Author, BookCopy, CopyStatus, Isbn, CatalogSearch, CatalogService,
BookRepository
  Member, MembershipType, MemberService, MemberRepository, FineAccount
  Loan, LoanStatus, LoanPolicy, LoanService, LoanRepository, DueDateCalculator
  Reservation, ReservationQueue, ReservationService, ReservationRepository
  NotificationTemplate, EmailSender, ReminderScheduler, NotificationLog
  LoginController, PasswordHasher, JwtIssuer, UserAccount, Role
  BookController, LoanController, MemberController,
LibrarianDashboardController, *Dto

"Borrow a book" (Lab-10 sequence diagram), messages between lifelines:
  Librarian -> LoanController      : POST /loans {memberId, copyId}
  LoanController -> LoanService    : borrow(memberId, copyId)
```

```
  LoanService -> MemberService     : canBorrow(memberId)         (checks fines,
limit)
  LoanService -> CatalogService    : isAvailable(copyId)
  LoanService -> ReservationService: hasPriorityHold(copyId, memberId)
  LoanService -> LoanPolicy        : dueDate(membershipType)
  LoanService -> LoanRepository    : save(loan)
  LoanService -> CatalogService    : markOnLoan(copyId)
  LoanService -> EmailSender       : sendLoanConfirmation(member, loan)


' diagrams/task1-components.puml (skeleton)
@startuml
skinparam componentStyle uml2
component "Loans" as Loans
component "Catalogue" as Catalogue
' TODO: the remaining 4-6 components

interface LoanApi
interface LoanQuery
interface CatalogLookup
' TODO: CopyAvailability, MemberLookup, ReservationApi, Notifier, AuthApi ...

LoanApi - Loans
LoanQuery - Loans
CatalogLookup - Catalogue
Loans ..> CatalogLookup : uses
' TODO: every required interface as C ..> I : uses
@enduml
```

EXPECTED OUTPUT

```
docs/components.md (excerpt)
| Component     | Classes                                                  | Owned
tables        |
|---------------|----------------------------------------------------------|--------
-------------|
| Loans         | Loan, LoanStatus, LoanPolicy, LoanService, LoanRepository,
DueDateCalculator | loan |
| Catalogue     | Book, Author, BookCopy, CopyStatus, Isbn, CatalogSearch,
CatalogService, BookRepository | book, author, book_copy |
...
| Component     | Provides                              | Requires
|
| Loans         | LoanApi, LoanQuery                    | MemberLookup,
CopyAvailability, ReservationApi, Notifier |
| Catalogue     | CatalogLookup, CopyAvailability       | (none)
|
```

```
| Notifications | Notifier                                  | (external: SMTP
provider)                      |

task1-components.png: 7 components, 8 lollipops, 9 dashed "uses" arrows; no
class names visible.
```

Show hints

- Controllers and DTOs all go into **Web UI**; they are the driving adapters of every other component.
- `FineAccount` is owned by Members (it changes with membership rules), even though Loans creates fines: Loans *requires* an operation like `MemberLookup.chargeFine`, or better publishes a `LoanOverdue` event.
- A "required interface with no provider" is fine when the provider is outside the system (SMTP server, campus SSO). Mark it *external*.
- If two components require each other you have found a cycle already; note it, Task 5 fixes it.

ACCEPTANCE CHECKLIST

- [ ] `components.md` groups every listed class into exactly one of 6-8 components and names the tables each owns.
- [ ] Every cross-component message of the two sequence diagrams is mapped to a named interface operation.
- [ ] Each required interface is provided by exactly one component or marked external.
- [ ] `task1-components.puml` renders to PNG with every interface shown once as a lollipop.

2

Draw the component diagram: ports, lollipops, sockets, assembly connectors

Easy **45 min**

GOAL

Turn the inventory from Task 1 into a proper UML 2.5 component diagram of the library as one `«subsystem»`: every requirement wired to its provision by an assembly connector, ports named, infrastructure and external systems outside the boundary, plus a dependency table.

STEPS

1. Create `diagrams/task2-library-components.puml`. Put all components inside `package "Campus Library" <<subsystem>> { ... }`. Give Loans two **ports**: `portin api` (LoanApi, LoanQuery) and `portout outbound` (everything it requires).
2. Replace every `C ..> I : uses` from Task 1 by the socket notation `C -( I` and keep `I - P` for the provider, so PlantUML draws ball-and-socket assembly connectors. Web UI must have sockets for `LoanApi`, `CatalogLookup`, `ReservationApi`, `MemberLookup` and `AuthApi`.
3. Add outside the subsystem: `database "library_db (PostgreSQL 17)"`, `cloud "Campus SMTP"`, `cloud "Campus SSO (OIDC)"`. Connect them with dashed `..>` dependencies labelled with the protocol (JDBC, SMTP, OIDC) from the component that owns the adapter.
4. Write `docs/dependency-table.md`: one row per connector, columns *From · To · Interface · Kind (assembly / dependency «use» / external) · Sync or async · Operations (count)*. Sort by *From*.
5. Check the diagram against three rules and note the result at the bottom of the table: (a) no dashed arrow between two business components, (b) every socket is wired, (c) no component reaches the database except through its own adapter (so 4-5 JDBC arrows, one per component that owns tables).
6. Render to PNG; then draw the same diagram once more by hand in draw.io or on paper using the notation from the deck (slide 4) and photograph or export it. Both versions go into `diagrams/`.

STARTER DIAGRAM TEXT

```
@startuml
skinparam componentStyle uml2
left to right direction

package "Campus Library" <<subsystem>> {
  component "Web UI" as UI
  component "Loans" as Loans {
    portin api
    portout outbound
  }
  component "Catalogue" as Cat
  component "Members" as Mem
  component "Reservations" as Res
  component "Notifications" as Not
  component "Identity" as Id
}
database "library_db\n(PostgreSQL 17)" as DB
cloud "Campus SMTP" as SMTP
cloud "Campus SSO\n(OIDC)" as SSO

interface LoanApi
```

```
interface LoanQuery
interface CatalogLookup
interface CopyAvailability
' TODO: MemberLookup, ReservationApi, Notifier, AuthApi

' provided (ball)
LoanApi - api
LoanQuery - api
CatalogLookup - Cat
CopyAvailability - Cat

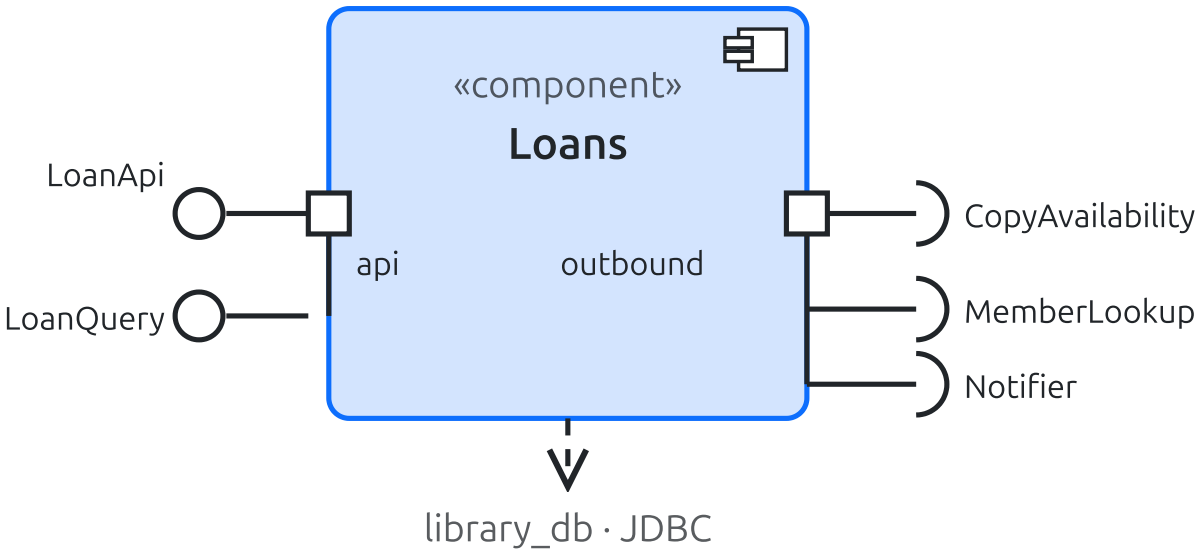
' required (socket) = assembly connectors
UI -( LoanApi
UI -( CatalogLookup
outbound -( CopyAvailability
' TODO: outbound -( MemberLookup, ReservationApi, Notifier ; UI -( ... ; Res -(
CopyAvailability

' infrastructure and external systems
Loans ..> DB : JDBC
Cat ..> DB : JDBC
Not ..> SMTP : SMTP
Id ..> SSO : OIDC
@enduml
```

EXPECTED OUTPUT

```
docs/dependency-table.md (excerpt)
| From  | To            | Interface        | Kind      | Sync  | Ops |
|-------|---------------|------------------|-----------|-------|-----|
| Loans | Catalogue     | CopyAvailability | assembly  | sync  | 3   |
| Loans | Members       | MemberLookup     | assembly  | sync  | 2   |
| Loans | Reservations  | ReservationApi   | assembly  | sync  | 1   |
| Loans | Notifications | Notifier         | assembly  | sync  | 2   |
| Loans | library_db    | -                | «use»     | sync  | -   |
| UI    | Loans         | LoanApi          | assembly  | sync  | 3   |
...
Rules: (a) OK  (b) OK  (c) OK, 5 JDBC arrows (Loans, Catalogue, Members,
Reservations, Identity)
```

Loans as it must look in task2 (PlantUML draws the same)



Show hints

- PlantUML socket syntax: `A -( I` draws A with a socket towards I; `I - B` attaches the ball to B. `A -(0- B` draws a full assembly connector in one line.
- Ports: `portin` / `portout` inside the component body; connect interfaces to the port name, not to the component.
- If the layout is a mess, try `top to bottom direction` or hidden edges (`Cat -[hidden]- Mem`) to align components.
- The database arrows carry a stereotype-free protocol label (`: JDBC`); do not draw a lollipop for the database, it is not a component you designed.

ACCEPTANCE CHECKLIST

- [ ] All components sit inside the «subsystem» package; database, SMTP and SSO are outside and reached by labelled dashed dependencies.
- [ ] Loans has an `api` port and an `outbound` port; every requirement is a socket wired to a lollipop (assembly connector).
- [ ] `dependency-table.md` has one row per connector with kind, sync/async and operation count, and the three rule checks.
- [ ] Both the PlantUML PNG and the hand-drawn version are in `diagrams/` and agree with each other.

3 Loans: from layered to hexagonal (ports and adapters)

Medium **45 min**

GOAL

Zoom into the Loans component. Start from the layered version you know from Chapter 02 and redraw it as a hexagon: the domain in the middle declaring **ports**, and **adapters** for REST, JPA and email at the edge. Justify the change in writing.

STEPS

1. Copy the layered starter below to `diagrams/task3-loans-layered.puml`, render it, and annotate on the PNG (or in a comment block) the two arrows that make the domain depend on infrastructure.
2. Create `diagrams/task3-loans-hexagonal.puml`. Inside a component `Loans` draw an inner `rectangle "Domain + Application"` holding `Loan`, `LoanPolicy`, `LoanService` and `DueDateCalculator`. Declare the **inbound ports** `LoanApi` and `LoanQuery` as `portin`, and the **outbound ports** `LoanRepository`, `CopyAvailability`, `MemberLookup`, `ReservationApi`, `Notifier` as `portout`.
3. Add the adapters as components outside the hexagon: `RestLoanAdapter` (driving, calls LoanApi), `JpaLoanRepository` (driven, implements LoanRepository), `EmailNotifierAdapter` (driven, implements Notifier), plus the three in-process adapters that call the other components' APIs. Every arrow must point **into** the domain or into a port.
4. List the Java packages that realise the picture in `docs/loans-hexagonal.md`: `edu.itc.library.loans.api` (inbound ports and event records), `edu.itc.library.loans.spi` (outbound ports), `edu.itc.library.loans.domain`, `edu.itc.library.loans.application`, `edu.itc.library.loans.adapter.web`, `adapter.persistence`, `adapter.email`. State for each package what it may import.
5. Write the justification (max one page) with at least five concrete arguments tied to this system, for example: unit-testing `LoanService` with a fake `CopyAvailability`; replacing SMTP by the campus messaging API; keeping Jakarta Persistence annotations out of `Loan`; extracting Notifications later (challenge); preserving the Chapter 08 domain model unchanged.
6. Name one thing that gets *worse* with hexagonal (more classes, mapping between entity and JPA model, indirection) and say why you accept it here.

STARTER: THE LAYERED LOANS

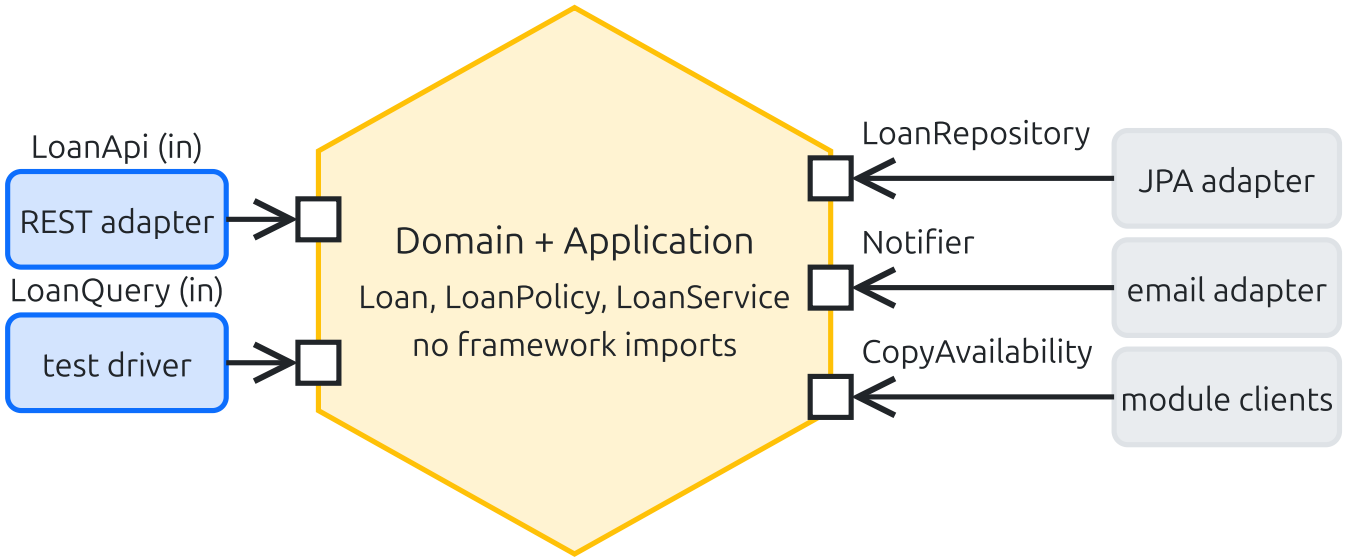
```
@startuml
skinparam componentStyle uml2
component "Loans (layered)" as Loans {
```

```
    rectangle "Presentation" as P {
      [LoanController]
      [LoanDto]
    }
    rectangle "Application" as A {
      [LoanService]
    }
    rectangle "Domain" as D {
      [Loan]
      [LoanPolicy]
      [DueDateCalculator]
    }
    rectangle "Infrastructure" as I {
      [JpaLoanRepository]
      [LoanEntity]
      [SmtpEmailSender]
    }
  }
  [LoanController] --> [LoanService]
  [LoanService] --> [Loan]
  [LoanService] --> [LoanPolicy]
  [LoanService] --> [JpaLoanRepository]
  [LoanService] --> [SmtpEmailSender]
  [Loan] --> [LoanEntity] : extends?!
  [JpaLoanRepository] --> [LoanEntity]
  @enduml
```

EXPECTED OUTPUT

```
  diagrams/task3-loans-hexagonal.puml (core)
  @startuml
  skinparam componentStyle uml2
  component "Loans" as Loans {
    portin LoanApi
    portin LoanQuery
    portout LoanRepository
    portout CopyAvailability
    portout MemberLookup
    portout ReservationApi
    portout Notifier
    hexagon "Domain + Application\nLoan, LoanPolicy, LoanService" as Core
  }
  [RestLoanAdapter] --> LoanApi
  [RestLoanAdapter] --> LoanQuery
  [JpaLoanRepository] .up.|> LoanRepository
  [EmailNotifierAdapter] .up.|> Notifier
  [CatalogueClient] .up.|> CopyAvailability
```

```
[MembersClient] .up.|> MemberLookup
[ReservationsClient] .up.|> ReservationApi
LoanApi --> Core
Core --> LoanRepository
Core --> Notifier
@enduml
' every arrow points inwards; Core has no outgoing arrow to an adapter
```



all arrows point inwards: adapters depend on ports, never the reverse

Show hints

- PlantUML has a `hexagon` shape (PlantUML 1.2023+); if your version lacks it, use `rectangle` and say so in the doc.
- The implementation arrow `.up.|>` (dashed with hollow triangle) is the UML "realizes" relation: adapter realizes port.
- The in-process adapters (`CatalogueClient`) are just Spring beans that call `CatalogLookup` from the Catalogue module; their job is to translate `BookCopyView` into the Loans domain type.
- Argument bank: testability, replaceability, dependency direction, domain purity, extraction path, parallel team work.

ACCEPTANCE CHECKLIST

- [ ] The layered diagram is rendered and its two domain-to-infrastructure arrows are identified.

- [ ] The hexagonal diagram shows 2 inbound and 5 outbound ports with REST, JPA and email adapters realising them; every arrow points inwards.
- [ ] `loans-hexagonal.md` maps the picture to the seven packages and states the allowed imports of each.
- [ ] The justification gives five system-specific arguments and one accepted drawback.

4

Map components to code: Maven modules, module-info, port/adapter, ArchUnit

Medium **60 min**

GOAL

Make the diagram compiler-enforced: a Maven reactor with one module per component (`library-catalog`, `library-loans`, `library-notifications`, `library-app`), a `module-info.java` per module mirroring provided/required interfaces, one real port/adapter pair, and an ArchUnit test that fails the build if Loans ever depends on web classes.

STEPS

1. Create `library/pom.xml` from the starter (packaging `pom`, parent `spring-boot-starter-parent:4.1.1`, `java.version` 25, the four `<modules>` in dependency order). Create the four module folders with their own `pom.xml`; `library-app` depends on the other three, `library-loans` depends on `library-catalog`, `library-notifications` depends on `library-loans` (it implements a Loans port). Run `mvn -q validate` and fix the reactor until it prints nothing.
2. Add `src/main/java/module-info.java` to each module. `edu.itc.library.loans` exports `..loans.api` and `..loans.spi`, requires `edu.itc.library.catalog`, `spring.context` and `spring.tx`, and `opens ..loans.application to spring.core, spring.beans`. `edu.itc.library.notifications` requires `edu.itc.library.loans` and `provides edu.itc.library.loans.spi.Notifier with ...LoggingNotifierAdapter`.
3. Implement the port/adapter pair: in loans, `spi/Notifier.java` (`void loanConfirmed(LoanConfirmation c)`, `void loanOverdue(OverdueNotice n)`) and `application/LoanService.java` that calls it after saving. In notifications, `LoggingNotifierAdapter implements Notifier` (logs a line; the SMTP variant is optional). In `library-app`, `NotificationsConfig` with a `@Bean Notifier` and a `@Profile("test")` fake.

4. Add a minimal `CopyAvailability` interface in `library-catalog` (`boolean isAvailable(CopyId id)`, `void markOnLoan(CopyId id)`) with an in-memory implementation, so `LoanService.borrow(...)` compiles against a real required interface from another module.
5. Write `library-app/src/test/java/edu/itc/library/ArchitectureTests.java` with the two rules from the starter. To prove the guard works, temporarily add `import org.springframework.web.bind.annotation.RestController;` and the annotation to `LoanService`, run `mvn -q verify`, capture the failure into `docs/archunit-failure.txt`, then remove the import and verify again (green).
6. Add a JUnit 6 unit test `LoanServiceTest` in `library-loans` that uses a hand-written fake `Notifier` and fake `CopyAvailability` (no Spring, no Mockito) and asserts that `borrow` marks the copy on loan and calls `loanConfirmed` once.

STARTER CODE

```
<!-- library/pom.xml -->
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
  </parent>
  <groupId>edu.itc.library</groupId>
  <artifactId>library</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>
  <properties><java.version>25</java.version></properties>
  <modules>
    <module>library-catalog</module>
    <module>library-loans</module>
    <module>library-notifications</module>
    <module>library-app</module>
  </modules>
  <dependencyManagement>
    <dependencies>
      <!-- TODO: archunit-junit5 1.4.1 (test) from the Setup section -->
    </dependencies>
  </dependencyManagement>
</project>


// library-loans/src/main/java/module-info.java
module edu.itc.library.loans {
    exports edu.itc.library.loans.api;      // provided: LoanApi, LoanQuery,
```

```
events
    exports edu.itc.library.loans.spi;      // required ports: Notifier, ...
    requires edu.itc.library.catalog;       // CopyAvailability lives there
    requires spring.context;
    requires spring.tx;
    // TODO: opens ..loans.application to spring.core, spring.beans;
}

// library-loans/src/main/java/edu/itc/library/loans/spi/Notifier.java
package edu.itc.library.loans.spi;

public interface Notifier {
    void loanConfirmed(LoanConfirmation confirmation);
    void loanOverdue(OverdueNotice notice);

    record LoanConfirmation(String memberEmail, String bookTitle,
                            java.time.LocalDate dueDate) {}
    record OverdueNotice(String memberEmail, String bookTitle,
                         long daysOverdue) {}
}

// library-
loans/src/main/java/edu/itc/library/loans/application/LoanService.java
package edu.itc.library.loans.application;

@Service
public class LoanService implements LoanApi {
    private final LoanRepository loans;
    private final CopyAvailability copies;      // from library-catalog
    private final Notifier notifier;            // port, implemented elsewhere

    public LoanService(LoanRepository loans, CopyAvailability copies,
                       Notifier notifier) {
        this.loans = loans; this.copies = copies; this.notifier = notifier;
    }

    @Override
    @Transactional
    public LoanId borrow(BorrowCommand cmd) {
        // TODO: if (!copies.isAvailable(cmd.copyId())) throw new
CopyNotAvailable(...)
        // TODO: var loan = Loan.open(cmd,
LoanPolicy.STANDARD.dueDateFrom(today))
        // TODO: loans.save(loan); copies.markOnLoan(cmd.copyId());
        // TODO: notifier.loanConfirmed(new Notifier.LoanConfirmation(...));
        return null;
    }
}
```

```
// library-notifications/src/main/java/module-info.java
module edu.itc.library.notifications {
    requires edu.itc.library.loans;
    requires org.slf4j;
    provides edu.itc.library.loans.spi.Notifier
        with edu.itc.library.notifications.LoggingNotifierAdapter;
}

// library-app/src/test/java/edu/itc/library/ArchitectureTests.java
@AnalyzeClasses(packages = "edu.itc.library",
                importOptions = ImportOption.DoNotIncludeTests.class)
class ArchitectureTests {

    @ArchTest
    static final ArchRule loans_never_depend_on_web =
        noClasses().that().resideInAPackage("..loans..")
            .and().resideOutsideOfPackage("..loans.adapter.web..")
            .should().dependOnClassesThat()
            .resideInAnyPackage("..web..", "org.springframework.web..",
                                "jakarta.servlet..");

    @ArchTest
    static final ArchRule modules_talk_through_api_and_spi_only =
        noClasses().that().resideOutsideOfPackage("..loans..")
            .should().dependOnClassesThat()
            .resideInAnyPackage("..loans.application..", "..loans.domain..");

    // TODO (optional): slices().matching("edu.itc.library.
(*)..").should().beFreeOfCycles()
}
```

EXPECTED OUTPUT

```
$ mvn -q verify                       # with @RestController on LoanService
[ERROR] Tests run: 2, Failures: 1, Errors: 0, Skipped: 0
[ERROR] ArchitectureTests.loans_never_depend_on_web
java.lang.AssertionError: Architecture Violation [Priority: MEDIUM] - Rule 'no
classes that
reside in a package '..loans..' and reside outside of package
'..loans.adapter.web..' should
depend on classes that reside in any package ['..web..',
'org.springframework.web..',
'jakarta.servlet..']' was violated (1 times):
Class <edu.itc.library.loans.application.LoanService> is annotated with
<org.springframework.web.bind.annotation.RestController> in (LoanService.java:0)
```

```
$ mvn -q verify                          # after removing the import
$ echo $?
0
$ jdeps -summary library-loans/target/library-loans-1.0.0.jar
edu.itc.library.loans -> edu.itc.library.catalog
edu.itc.library.loans -> java.base
edu.itc.library.loans -> spring.context
edu.itc.library.loans -> spring.tx
```

Show hints

- Automatic module names: Spring 7 jars declare `spring.context`, `spring.tx`, `spring.beans`, `spring.core`; SLF4J is `org.slf4j`. If `mvn compile` says "module not found", check the jar's `Automatic-Module-Name`.
- Keep `library-app` as the only module with `@SpringBootApplication`; it needs `requires edu.itc.library.loans;` etc. and `opens edu.itc.library to spring.core;`.
- The ArchUnit dependency goes into `library-app` only; the rules analyse the whole `edu.itc.library` package tree because all modules are on its test classpath.
- The failure text must come from a real run: graders diff it against the rule names.
- A fake in the unit test is a tiny class: `class FakeNotifier implements Notifier { int confirmed; public void loanConfirmed(...) { confirmed++; } ... }`.

ACCEPTANCE CHECKLIST

- [ ] The four-module reactor builds with `mvn -q verify` from a clean checkout; each module has a `module-info.java` whose exports/requires match the Task 2 diagram.
- [ ] `Notifier` port in loans, `LoggingNotifierAdapter` in notifications (`provides … with`), wired by a `@Configuration` in library-app with a test-profile fake.
- [ ] `docs/archunit-failure.txt` contains the real failure of `loans_never_depend_on_web`, and the final build is green.
- [ ] `LoanServiceTest` runs without Spring using hand-written fakes for both ports.

5

Validate and document: dependency review, ADRs, C4 container diagram

Hard **60 min**

GOAL

Review a flawed component diagram handed to you by "the previous team", find the cycle and the boundary violations, fix them with interface inversion, prove the fix with tooling, and record the decisions as ADRs and a C4 container diagram.

STEPS

1. Render the flawed diagram below (`diagrams/task5-flawed.puml`). In `docs/review/dependency-review.md` list every problem you find with the rule it breaks (ADP cycle, SDP direction, content coupling into internals, shared table, unwired socket). Expect at least four findings, including one cycle of length 2 and one of length 3.
2. Draw the dependency graph as a table of edges and compute for each component *Ca*, *Ce* and *I = Ce / (Ca + Ce)*. Mark every edge that points from a more stable to a less stable component (SDP violation).
3. Fix the diagram in `diagrams/task5-fixed.puml` using **interface inversion**: Loans publishes `LoanEvents` (`LoanCreated`, `LoanReturned`, `LoanOverdue`); Notifications and Members require that interface instead of being called; Catalogue must not know Loans at all. Every remaining arrow must go towards a component with lower *I*.
4. Apply the fix in code: add `edu.itc.library.loans.api.LoanEvents` (a record per event) published with `ApplicationEventPublisher` from `LoanService`, and an `@EventListener` (or Spring Modulith `@ApplicationModuleListener`) in `library-notifications`. Delete the direct `Notifier.loanConfirmed` call if you now use the event; keep `Notifier` for the overdue reminder, or document why you removed it.
5. Run `mvn -q dependency:analyze -DfailOnWarning=true` and `jdeps -summary` on the jars; paste both outputs into the review and fix every "used undeclared" and "unused declared" warning. Enable the ArchUnit `beFreeOfCycles()` rule from Task 4.
6. Write `docs/adr/ADR-001-modular-monolith.md` (why one deployable with JPMS/Maven boundaries instead of microservices now) and `docs/adr/ADR-002-loan-events.md` (why events replace direct calls from Loans to Notifications/Members) using the template. Then write `diagrams/task5-c4-container.puml` with the PlantUML C4 library: `Person` Librarian and Student, `System_Boundary`, one `Container` for the Spring Boot app listing its modules, `ContainerDb`, and the external SSO and SMTP systems with labelled `Rel`s.

STARTER: THE FLAWED DIAGRAM

```
@startuml
' diagrams/task5-flawed.puml : as inherited from the previous team
skinparam componentStyle uml2
component "Web UI" as UI
component Loans
component Catalogue
component Members
component Notifications
```

```
component Reservations
database library_db

UI ..> Loans
UI ..> Catalogue
UI ..> Members
Loans ..> Catalogue : isAvailable, markOnLoan
Loans ..> Members : canBorrow
Loans ..> Notifications : sendLoanConfirmation
Notifications ..> Loans : LoanRepository.findDueTomorrow()
Members ..> Loans : LoanService.activeLoans(member)
Catalogue ..> Reservations : nextInQueue(copy)
Reservations ..> Loans : LoanQuery
Loans ..> library_db
Catalogue ..> library_db
Members ..> library_db
Notifications ..> library_db : reads table loan
@enduml
```

```
# ADR-NNN: <title>
* Status: Proposed | Accepted | Superseded by ADR-MMM
* Date: 2026-MM-DD · Deciders: <names>

## Context
What forces are at play (functional, quality goals, constraints, findings from
the review)?

## Decision
The change we are making, in one paragraph, with the diagram element it affects.

## Consequences
+ positive · - negative · = neutral (at least two of each)

## Alternatives considered
1. ... (why rejected)
2. ...
```

EXPECTED OUTPUT

```
docs/review/dependency-review.md (excerpt)
| # | Finding                                                | Rule            |
Fix                              |
|---|--------------------------------------------------------|-----------------|--
---------------------------------|
| 1 | Loans -> Notifications -> Loans                        | ADP (cycle, 2)  |
Loans publishes LoanEvents       |
```

```
| 2 | Loans -> Members -> Loans                              | ADP (cycle, 2)  |
Members listens to LoanEvents    |
| 3 | Loans -> Catalogue -> Reservations -> Loans           | ADP (cycle, 3)  |
Reservations requires LoanEvents; Catalogue drops nextInQueue |
| 4 | Notifications reads Loans' LoanRepository + table loan | content coupling,
shared table | event carries due date |
| 5 | I(Loans)=0.57 but 3 components depend on it           | SDP             |
after fix: I(Loans)=0.33         |

$ mvn -q dependency:analyze -DfailOnWarning=true      # after fix: no output,
exit 0
$ jdeps -summary library-notifications/target/*.jar
edu.itc.library.notifications -> edu.itc.library.loans      (api package only)
edu.itc.library.notifications -> java.base
edu.itc.library.notifications -> spring.context

diagrams/task5-c4-container.puml (excerpt)
@startuml
!include <C4/C4_Container>
Person(librarian, "Librarian")
Person(student, "Student")
System_Boundary(lib, "Campus Library") {
  Container(app, "Library Application", "Spring Boot 4.1, Java 25",
    "modules: loans, catalog, members, reservations, notifications, identity")
  ContainerDb(db, "library_db", "PostgreSQL 17", "one schema per module")
}
System_Ext(sso, "Campus SSO", "OIDC")
System_Ext(smtp, "Campus SMTP", "email")
Rel(librarian, app, "Manages loans", "HTTPS")
Rel(student, app, "Searches, reserves", "HTTPS")
Rel(app, db, "JDBC")
Rel(app, sso, "Authenticates", "OIDC")
Rel(app, smtp, "Sends reminders", "SMTP")
@enduml
```

Show hints

- Count edges among the six business components only (ignore the database) for Ca/Ce. A cycle is any path that returns to its start; list all three before fixing.
- Interface inversion means the *callee* defines the interface the *caller* once owned. Here the stable module (Loans) publishes; the unstable ones (Notifications, Members, Reservations) subscribe.
- `Catalogue ..> Reservations : nextInQueue` is the wrong direction: Reservations knows about copies, Catalogue must not know about holds. Move the check into Loans (it already requires ReservationApi) or into an event listener.
- Spring Modulith users: `ApplicationModules.of(LibraryApplication.class).verify()` reports the cycles for you; still write them down by hand first.

- C4-PlantUML is bundled in the standard library: `!include <C4/C4_Container>` works offline with a recent plantuml.jar.

ACCEPTANCE CHECKLIST

- [ ] The review names at least four findings with the rule broken, the Ca/Ce/I table and the three cycles.
- [ ] `task5-fixed.puml` is acyclic, uses a `LoanEvents` interface, and every edge points to a component with lower instability.
- [ ] The event is implemented in code; `dependency:analyze`, `jdeps` and the `beFreeOfCycles` rule all pass and their output is in the review.
- [ ] Two complete ADRs and a rendered C4 container diagram with people, container, database and both external systems.

★

Challenge: extract Notifications into a service behind a message broker

Hard **90–120 min · optional, extra credit**

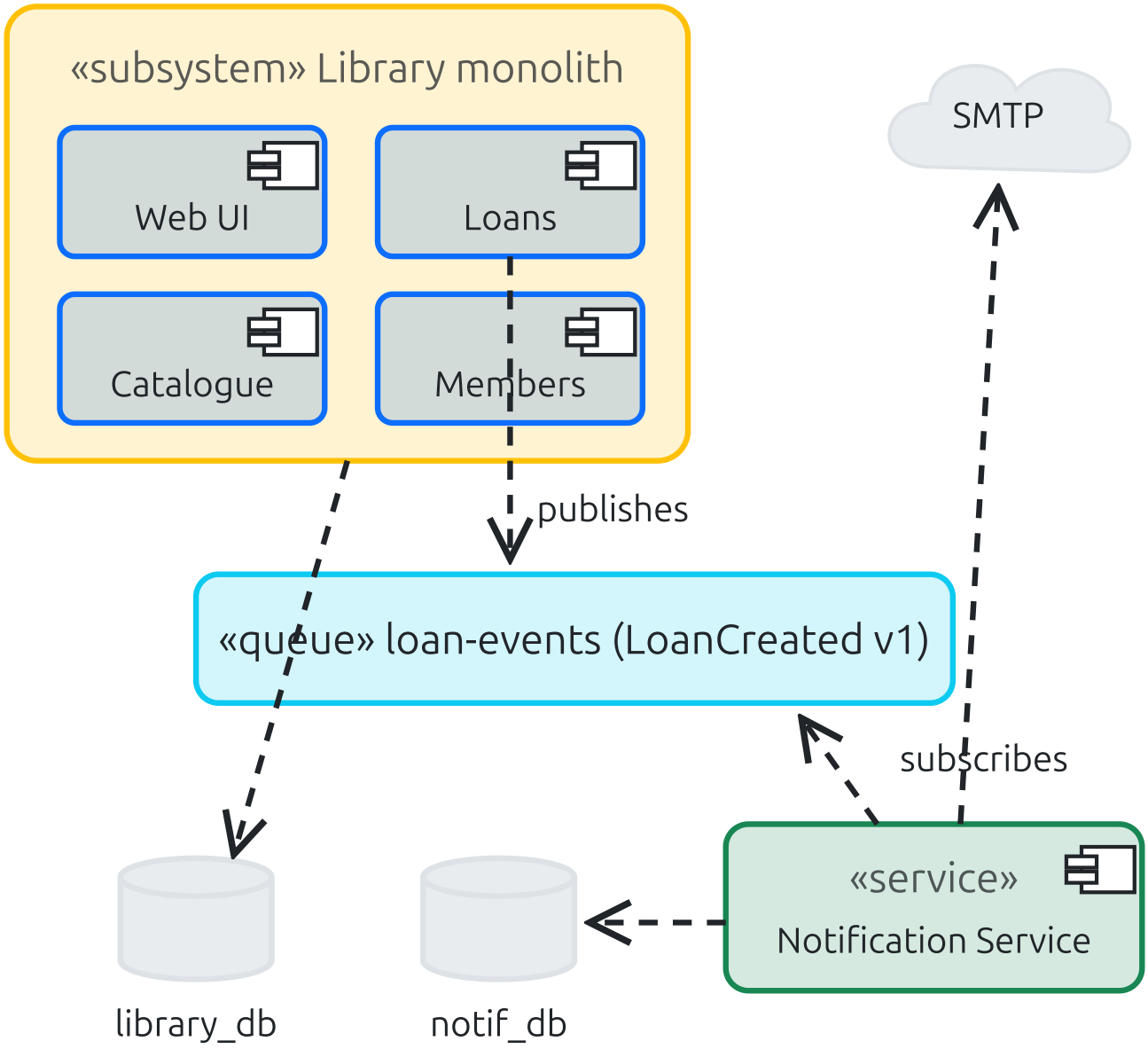
SCENARIO

The campus IT department wants reminder emails sent by a shared notification service used by several systems. Notifications must leave the monolith and run as its own deployable, fed by `LoanEvents` over a message broker. The library monolith must keep working (and its tests keep passing) whether or not the new service is running.

REQUIREMENTS

1. **Before/after diagrams.** `diagrams/challenge-before.puml` (the Task 5 fixed monolith) and `diagrams/challenge-after.puml`: monolith without Notifications, a `queue "loan-events"`, and `component "Notification Service" <<service>>` with its own database and the SMTP dependency. The `LoanEvents` interface must appear in both, once as a Java interface, once as an event contract on the queue.
2. **Event contract.** `contracts/loan-created.schema.json`: a JSON Schema (draft 2020-12) for `LoanCreated` with `eventId` (uuid), `occurredAt` (date-time), `loanId`, `memberEmail`, `bookTitle`, `dueDate` (date) and a `schemaVersion`; all required, `additionalProperties: false`. Do the same for `LoanOverdue`.

3. **Publishing.** Either Spring Modulith event externalisation (`@Externalized("loan-events::loan.created")` on the event record plus `spring-modulith-events-kafka` or `-amqp` and the JDBC event publication registry) or a hand-written `KafkaTemplate`/`RabbitTemplate` outbox sketch. Name the topic/exchange and the routing key in an ADR-003.
4. **Migration plan.** `docs/migration-plan.md`: a table with at least six steps (dual-write, shadow consumer, cut-over, decommission ...), each with *Step · Risk · Rollback · Verification*. State what happens to in-flight reminders during cut-over.
5. **Integration test.** `LoanFlowWithoutNotificationsIT` in `library-app`: `@SpringBootTest` with the broker client replaced by `@MockitoBean` (or Modulith's `Scenario` API / `AssertablePublishedEvents`); borrow a copy, assert the loan is persisted, assert exactly one `LoanCreated` was published with the schema's fields, and assert no SMTP call happened in the monolith.
6. **Deployment teaser.** One paragraph and a sketch of where the new service runs (container, node, port), to be completed in Chapter 12.



SKELETON

```
// library-loans/src/main/java/edu/itc/library/loans/api/LoanCreated.java
package edu.itc.library.loans.api;

import org.springframework.modulith.events.Externalized;

@Externalized("loan-events::loan.created")   // topic::routing key
public record LoanCreated(UUID eventId, Instant occurredAt, String loanId,
                          String memberEmail, String bookTitle,
                          LocalDate dueDate, int schemaVersion) {
    public static LoanCreated of(Loan loan, String email, String title) {
        return new LoanCreated(UUID.randomUUID(), Instant.now(),
                               loan.id().value(), email, title,
                               loan.dueDate(), 1);
```

```
    }
}

// library-app/src/test/java/edu/itc/library/LoanFlowWithoutNotificationsIT.java
@SpringBootTest
@ActiveProfiles("test")
@ApplicationModuleTest(mode =
ApplicationModuleTest.BootstrapMode.ALL_DEPENDENCIES)
class LoanFlowWithoutNotificationsIT {

    @Autowired LoanApi loans;
    @Autowired LoanQuery query;
    @MockitoBean KafkaTemplate<String, Object> kafka;   // broker stubbed
    @MockitoBean JavaMailSender mail;                    // must never be used

    @Test
    void borrowingPublishesLoanCreatedAndNeedsNoNotificationService(
            AssertablePublishedEvents events) {
        var id = loans.borrow(new BorrowCommand(new MemberId("m-1"),
                                                new CopyId("c-42")));

        assertThat(query.byId(id)).isPresent();
        assertThat(events).contains(LoanCreated.class)
            .matching(e -> e.loanId().equals(id.value())
                        && e.schemaVersion() == 1);
        verifyNoInteractions(mail);
        // TODO: assert the externalised payload validates against
        //       contracts/loan-created.schema.json (networknt json-schema-
validator)
    }
}



{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://itc.edu/library/contracts/loan-created/v1",
  "title": "LoanCreated",
  "type": "object",
  "required": ["eventId", "occurredAt", "loanId", "memberEmail",
               "bookTitle", "dueDate", "schemaVersion"],
  "additionalProperties": false,
  "properties": {
    "eventId":       { "type": "string", "format": "uuid" },
    "occurredAt":    { "type": "string", "format": "date-time" },
    "loanId":        { "type": "string" },
    "memberEmail":   { "type": "string", "format": "email" },
    "bookTitle":     { "type": "string", "minLength": 1 },
    "dueDate":       { "type": "string", "format": "date" },
```

```
      "schemaVersion": { "const": 1 }
    }
  }
```

EXPECTED OUTPUT

```
$ mvn -q verify
[INFO]
LoanFlowWithoutNotificationsIT.borrowingPublishesLoanCreatedAndNeedsNoNotificati
onService  OK
$ java -jar plantuml.jar -tpng diagrams/challenge-*.puml   → challenge-
before.png, challenge-after.png

docs/migration-plan.md (excerpt)
| Step | Action                                  | Risk                    |
Rollback              | Verification                          |
| 1    | Publish LoanCreated to broker (dual)     | broker down blocks loan?|
outbox = async, no    | event count == loans created (24 h)   |
| 2    | Deploy Notification Service, shadow mode | duplicate emails        |
flag notif.send=false | logs show 1 consume per event         |
| 3    | Cut over: monolith stops sending mail    | missed reminders        |
re-enable in-process  | zero SMTP calls from monolith         |
| 4    | Remove library-notifications module      | build breaks            |
git revert            | mvn verify, jdeps shows no edge       |
| 5    | Replay events for in-flight reminders    | double send             |
idempotent eventId    | dedup table hit count                 |
| 6    | Decommission Notifier port               | -                       | -
| ArchUnit: no class implements Notifier|
```

Show hints

- Modulith externalisation needs `spring-modulith-events-api` (the annotation), `spring-modulith-starter-jdbc` (publication registry, guarantees at-least-once) and `spring-modulith-events-kafka`; in the test profile leave Kafka out and the registry alone proves publication.
- Without Modulith: an `@TransactionalEventListener(phase = AFTER_COMMIT)` that writes to an `outbox` table, and a scheduled relay that sends to the broker. Sketch is enough; the test stubs the relay.
- The "after" diagram must not contain a socket on the monolith for Notifier any more: the contract is the queue plus schema, drawn as `queue` with a note holding the schema id.
- Migration risk to think about: emails for loans created *before* the cut-over but due *after* it; the replay step or a one-off backfill job handles them.

ACCEPTANCE CHECKLIST

- [ ] Before/after component diagrams rendered; the after version has a «queue», a «service» with its own database, and no Notifier socket on the monolith.
- [ ] JSON Schemas for `LoanCreated` and `LoanOverdue` with all required fields, formats and `additionalProperties: false`; ADR-003 names topic and routing key.
- [ ] Publishing implemented (Modulith externalisation or outbox sketch) and the integration test passes with the broker and mail sender stubbed.
- [ ] Migration plan with six or more steps, each with risk, rollback and verification, and the in-flight reminder problem addressed.

Grading rubric

Component	Points	Full marks when…
Task 1 · Identify components and interfaces	10	6-8 cohesive components, every class placed once, every cross-component message mapped to a named interface, PlantUML rendered.
Task 2 · Component diagram	15	Correct UML 2.5 notation (ports, lollipops, sockets, assembly connectors), subsystem boundary, external systems outside, complete dependency table with rule checks.
Task 3 · Hexagonal Loans	20	Layered flaws identified; hexagon with 2 inbound / 5 outbound ports and three adapter kinds; package mapping; five arguments plus one drawback.
Task 4 · Map to code	20	Four-module reactor builds; module-info mirrors the diagram; port/adapter pair wired by configuration; ArchUnit failure captured then green; Spring-free unit test.
Task 5 · Validate and document	25	All cycles and violations found with metrics; fix by interface inversion in diagram and code; tool outputs clean; two complete ADRs; C4 container diagram.
Diagram / code quality	10	Consistent names across diagrams, docs and code; sources + PNG for every diagram; commits per task; README explains how to render and build.
★ Challenge (extra credit)	+20	All four acceptance items ticked; integration test passes with broker stubbed; migration plan realistic.

Component	Points	Full marks when…
Total	**100 (+20)**	

Automatic deductions: −5 per dashed «use» arrow between two business components that should be a named interface (max −15); −10 if the final diagram or the code still contains a dependency cycle; −10 if a Loans domain/application class imports web, servlet or JPA types; −5 per diagram submitted as image only (no source text); −10 if `module-info.java` is missing from any of the four modules; −5 if the ArchUnit failure text was written by hand instead of captured from a run.

Submission

1. Repository layout:

```
lab-11/
├── README.md                    how to render diagrams and build; one
paragraph per task
├── diagrams/
│   ├── task1-components.puml / .png
│   ├── task2-library-components.puml / .png   + task2-handdrawn.png
│   ├── task3-loans-layered.puml / .png
│   ├── task3-loans-hexagonal.puml / .png
│   ├── task5-flawed.puml / task5-fixed.puml / .png
│   ├── task5-c4-container.puml / .png
│   └── challenge-before.puml / challenge-after.puml / .png      (challenge)
├── docs/
│   ├── components.md · dependency-table.md · loans-hexagonal.md · archunit-
failure.txt
│   ├── review/dependency-review.md
│   ├── adr/ADR-001-modular-monolith.md · ADR-002-loan-events.md · ADR-003-
*.md (challenge)
│   └── migration-plan.md
(challenge)
├── contracts/loan-created.schema.json · loan-overdue.schema.json
(challenge)
└── library/                     Maven reactor: pom.xml + library-catalog,
library-loans,
                                 library-notifications, library-app (each
with module-info.java)
```

2. Verification that must pass from a clean checkout:

```
cd lab-11
java -jar plantuml.jar -tpng diagrams/*.puml          # every .puml renders
without error
cd library
mvn -q verify                                         # unit + ArchUnit (+
IT for the challenge)
mvn -q dependency:analyze -DfailOnWarning=true        # no used-undeclared /
unused-declared
jdeps -summary library-loans/target/library-loans-1.0.0.jar
```

3. Tag each task (`task-1` … `task-5`, `challenge`). Branch `lab-11`, merge request titled `Lab-11 – <your name>`; the description lists which acceptance items you ticked and links the rendered PNGs.

Back to Chapter 11 slides | Retake the chapter quiz | ← Lab-10